Upstream: what YARG's own rules say, and what we are asking them

The project's aim is that work here be upstreamable rather than a fork that diverges forever. This document records what upstream's process actually is — read from their documentation rather than assumed — and what we have asked them.

Their process, from `CONTRIBUTING.md`

- **Ask on Discord before building a feature.** Their words: "*It's recommended that you ask there before working on a new feature, in case someone is already working on a feature/change.*" Discord: https://discord.gg/sqpu4R552r. Task tracker: https://yarg.youtrack.cloud/agiles/147-7/current.
- **PRs target `dev`. A PR based on `master` will not be accepted.**
- Every feature falls in one of six tiers, and the tier decides whether a PR is even looked at:

Tier	What it means for a PR
In-Development	Do not PR without coordination — they are already on it
Planned	PRs welcome
Stretch Goal	Discouraged, but experimenting and sharing progress is invited, and "*large progress on a stretch goal may promote it to a higher tier*"
Eligible	PRs welcome; they will not build it themselves
Problematic	Heavily discouraged; big headachey impacts elsewhere
Out of Scope	"*Do NOT PR these features. Your PR will immediately be denied*"

Their Out of Scope examples include CON Decryption. That is worth noting: this project independently made CON/mogg decryption a permanent non-goal on legal grounds, and upstream has independently ruled it out on their own. Two of our three permanent non-goals are

things upstream would reject on sight. We are not asking them for anything they have already refused.

Which tier a remote song source falls into is not published, and none of the tier examples resemble it. That is the first question to ask, and it is exactly the question their contributing guide tells contributors to ask.

What we searched before asking

Nothing found in their issues proposes loading songs from a remote server. The nearest is **#860, "[FR] Built in Web Server for Song Queing/Search from external device"** — a *control plane* (search and queue from a phone) rather than a content source. Worth citing so it is clear we looked, and worth not conflating with what we are proposing.

But it is not merely adjacent, and that is worth saying in the post. #860 has been open since August 2024, unlabelled, with a comment pointing at a second Discord proposal that adds up/down votes on the queue — so there is demand and nobody has built it. The reason nobody has is in the request's own words: "*whilst YARG is running*". Nothing outside the game can reach a running client. **A remote song source is the thing that could** — the same channel that fetches songs can carry a queue. So this proposal is not novel work competing for their attention; it is the missing half of a request they already have.

Also searched and not found: any issue or PR about shared libraries, syncing songs between machines, or a network song source. `#1030 "Add support for user-supplied song sources"` is about **source icons**, not sources of songs.

The shape of the ask, and why it is smaller than it sounds

The server already hands out **plain `.sng` files that unmodified YARG reads natively** — that is the design commitment in ADR-001, and it has been demonstrated on two machines, two operating systems and two CPU architectures. So we are not asking upstream to support a protocol in order to play our songs. They already can.

What a player cannot do today is get those songs **without running a separate sync tool**. So the upstream ask is narrow: a way for YARG to discover and fetch songs from a URL. And there may be a smaller version of it still — `SongEntry` is abstract, but `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` all assume a local path, so the general capability underneath is *a song entry whose bytes do not come from the local filesystem*. That is a capability rather than a feature about our server, and it may be an easier thing for them to want.

This is reconnaissance, not a design. The real design belongs in an ADR once we know which tier this sits in and whether anyone upstream is already on it.

Draft: the Discord post

Not yet sent. Jay sends it, under his own account; nothing goes out without him.

> Hey folks — I've been building a self-hosted song server for YARG and wanted to ask about it here before going any further, per CONTRIBUTING.
>
> The short version: it's a small Go server that indexes a song folder and hands out plain `.sng` files. Nothing about it is a game modification — unmodified YARG reads what it serves, and I've had that running on two machines and two CPU architectures. Right now a little sync client pulls songs into a folder and YARG scans them like any other folder.
>
> What I'd like to ask about is the obvious next step: YARG being able to point at a server URL and browse/fetch from it directly, instead of needing a separate tool. Two questions:
>
> 1. Which tier does that fall into? It doesn't look like any of the examples in CONTRIBUTING, and I couldn't find an existing issue for it — the closest is #860, which is search/queue from a phone rather than a source of songs.
> 2. Is anyone already working on it? Happy to stay out of the way if so.
>
> To be clear on where I stand: I'm building it in my fork either way, so this isn't a request for anyone to do work — I'd just rather build it in a shape you'd consider than find out later it was never going to fit. And I'm not asking for anything you've ruled out: no CON decryption, no touching `songcache.bin`, and no distributing copyrighted audio. The server serves what the operator already has.
>
> One more thing, in case it makes the idea more interesting rather than less: the same channel would carry a **queue**. #860 has been open since 2024 asking for search-and-queue from a phone while YARG is running, and as far as I can tell it's unbuilt because nothing outside the game can reach a running client. A remote source would be the thing that could. I'm not proposing that part now — just noting the two are the same plumbing.
>
> One thing I noticed while reading `YARG.Core`: `SongEntry` is abstract, but `ActualLocation`, `SortBasedLocation` and `GetLastWriteTime()` all assume a local path. If a remote source is ever interesting to you, that's probably the seam — a song entry whose bytes aren't on disk. Curious whether that's been considered.
>
> Everything's LGPL-3.0-or-later, same as YARG: https://github.com/Coffeehedake/yarg-song-server

One rule about that link

The post links the public GitHub mirror, never `gitlab.badassium.com`. Origin is a GitLab instance on a home server; GitHub is a downstream, force-overwritten mirror that exists precisely so there is something public to point at. Pasting the origin URL into a Discord with a few thousand strangers in it would advertise home infrastructure for no benefit — the mirror carries identical content.

Verified before this draft went anywhere: `Coffeehedake/yarg-song-server` is public, LGPL-3.0, and its last push matches the last push to origin, so the mirror is live rather than a stale snapshot from months ago.

What to do with the answer

- **In-Development or someone is on it** — stop, and offer what we have to whoever is.
- **Planned or Eligible** — design toward a PR from the start, and target `dev`.
- **Stretch Goal** — build it in the fork and share progress; their own rule says large progress can promote a tier.
- **Problematic or Out of Scope** — build it in the fork for ourselves, drop the upstreaming goal for this feature specifically, and say so plainly in the ROADMAP rather than leaving a dead aspiration in it.

Their answer does not block Phase 3. It shapes it.
